

ParityLab proposal

Author: A Aswanth Raj

Title

Loss-Adaptive Packet-Level Parity FEC with Selective Repeat ARQ for Reliable Data Transfer over Lossy Networks

Abstract

Retransmission recovers lost packets but can increase delivery time, particularly when propagation delay is high and the sender's window is bounded. Forward error correction (FEC) sends repair packets that can reconstruct some losses before a retransmission arrives. Fixed redundancy introduces a tradeoff between bandwidth cost and recovery capability as network conditions change. This project implements a hybrid scheme that estimates raw packet loss from receiver feedback and selects a protection level for each block: no parity, one XOR parity packet with an adaptive block size, or two-dimensional row and column parity. Selective Repeat automatic repeat request (ARQ) recovers erasures that parity cannot repair. A Python event-driven emulator compares the scheme with Go-Back-N, plain Selective Repeat, and Selective Repeat with fixed XOR parity under independent, burst, and changing loss. The evaluation measures goodput, completion time, retransmissions, packet recovery delay, parity overhead, and FEC recovery ratio. A three-process localhost UDP transport verifies recovered file bytes for all four sliding-window schemes. Performance improvements are hypotheses to be tested rather than guaranteed outcomes.

Existing work

RFC 8681 [1] specifies sliding-window random linear FEC codes, establishing a modern reference for the relationship between protection and decoding latency. Golaghazadeh, Coulombe, and Robert [2] analyze residual packet loss for two-dimensional row/column parity, including the effect of matrix dimensions. FIEC [3] studies application-aware reliability choices inside QUIC, including bulk transfer and file transfers with restricted buffers. RFC 9002 [4] provides contemporary context for acknowledgment-based loss detection, retransmission timing, and congestion control; it is not a specification of the textbook Go-Back-N or Selective Repeat baselines. RFC 9265 [5] explains the interaction between FEC and congestion signals. A 2024 Internet-Draft [6] proposes loss-dependent redundancy for delay-sensitive QUIC connections. A 2026 preprint by Tsubaki et al. [7] evaluates adaptive FEC for Multipath QUIC datagrams over cellular paths. These sources establish that adaptive FEC is an existing research area. The contribution here is a transparent lab implementation and controlled comparison of simple parity modes paired with reliable retransmission.

Gap identified

1. Retransmission-only baselines incur loss-detection and resend delays. A bounded window can limit utilization on links with a large bandwidth-delay product. The exact penalty depends on the timer and channel conditions.
2. A fixed parity configuration has a fixed redundancy cost, while its ability to repair erasures depends on loss patterns. This project tests whether switching among simple parity modes improves the latency/overhead tradeoff under selected conditions.
3. The lab needs a reproducible comparison that uses the same payload, link settings, window, and seed set for all four schemes; counts parity and retransmission cost; and preserves raw loss feedback even when decoding succeeds.
4. An independent-loss estimate does not describe burst structure. The experiments must report when a controller misses its model target, when protection is infeasible, and when added parity reduces goodput.

Work proposed

1. Build a Python emulator with serialization delay, propagation delay, configurable bandwidth, Bernoulli loss, Gilbert-Elliott burst loss, lost acknowledgments, and changing loss phases.
2. Implement textbook Go-Back-N and Selective Repeat baselines with bounded windows and explicit timers; add fixed XOR parity to Selective Repeat.
3. Encode equal-length packet payloads using XOR and row/column parity. Pad the final packet and preserve the original file length. Decode by repeatedly repairing equations with one missing packet.
4. Estimate original data-packet erasures before FEC using an exponentially weighted moving average of receiver reports. Choose the least parity overhead among candidate configurations whose modeled probability of any unrepaired data in a block is at most 1%, accounting for lost repair packets. Use an exact independent-erasure calculation for small grids. If none meets the target, report infeasibility and select the lowest modeled failure probability.
5. Recover residual losses through Selective Repeat and verify the complete file bytes and SHA-256 digest. Reliability is conditional on eventual packet delivery, a working acknowledgment path, and the configured retry/event budget.
6. Evaluate a 0-20% independent-loss sweep, burst loss, and a changing-loss transfer with multiple seeds. Report means and sample standard deviations; examine sensitivity to delay, bandwidth, window, timer, and burst length before generalizing.

Metrics to be evaluated

- Goodput: original file bits divided by receiver completion time.
- File transfer completion time, with sender confirmation time reported separately.
- Number of retransmitted data packets.
- Mean and 95th percentile packet recovery delay, measured from first transmission until bytes become available at the receiver; this differs from in-order application delivery delay.
- Parity payload bytes divided by original file bytes, plus total modeled forward overhead.
- FEC recovery ratio: original data-packet losses repaired before any retransmission is sent, divided by original data-packet losses.

Final output

A Python implementation of the emulator and all four schemes; separate localhost sender, receiver, and impairment-proxy processes; reproducible experiment records; a comparison table; plots of goodput, retransmissions, overhead, completion time, and packet recovery delay; a recovery-delay cumulative distribution; and a controller mode-switching time series. The completed lab also includes impaired metadata and in-order delivery, risk calibration, uncertainty and burst-aware controller options, controlled sensitivity and equal-overhead comparisons, and a shared-bottleneck teaching congestion model. The interactive interface provides recorded replay and actual socket verification. The accompanying final report documents current evidence and limitations; Internet deployment remains outside the scope.

References (2020-2026)

- [1] V. Roca and B. Teibi, "Sliding Window Random Linear Code (RLC) Forward Erasure Correction (FEC) Schemes for FECFRAME," IETF RFC 8681, January 2020. DOI: 10.17487/RFC8681. [RFC Editor](#).
- [2] F. Golaghazadeh, S. Coulombe, and J. M. Robert, "Residual packet loss rate analysis of 2-D parity forward error correction," *Signal Processing: Image Communication*, vol. 102, article 116597, March 2022. DOI: 10.1016/j.image.2021.116597. [Publisher](#).
- [3] F. Michel, A. Cohen, D. Malak, Q. De Coninck, M. Medard, and O. Bonaventure, "FIEC: Enhancing QUIC with application-tailored reliability mechanisms," arXiv:2208.07741, August 2022. DOI: 10.48550/arXiv.2208.07741. [Author preprint](#).
- [4] J. Iyengar and I. Swett, Eds., "QUIC Loss Detection and Congestion Control," IETF RFC 9002, May 2021. DOI: 10.17487/RFC9002. [RFC Editor](#).
- [5] N. Kuhn, E. Lochin, F. Michel, and M. Welzl, "Forward Erasure Correction (FEC) Coding and Congestion Control in Transport," IETF RFC 9265, July 2022. DOI: 10.17487/RFC9265. [RFC Editor](#).
- [6] D. Moskvitin, E. Onegin, R. Huang, H. Luo, and Q. Chen, "Adaptive Forward Erasure Correction for Delay-Sensitive QUIC Connections," Internet-Draft draft-dmoskvitin-quic-adaptive-fec-00, May 6, 2024. Work in progress; this cited version expired November 7, 2024 and is not an adopted standard. [Versioned draft](#).
- [7] T. Tsubaki, S. Anno, S. Komatsu, T. Torii, and T. Tojo, "Scheduler-Agnostic Adaptive-FEC for MPQUIC: Field Evaluation over Commercial Cellular Paths," arXiv:2607.14482v1, July 16, 2026. DOI: 10.48550/arXiv.2607.14482. Cited as an author preprint. [Author preprint](#).